Algoritmos Distribuidos
Recorrido de Redes

Andrés Sanoja
Claudia León

Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 – Marzo 2009



Objetivos

- Comprender y describir el problema de recorrido de redes
- Comparar diferentes soluciones para el recorrido de redes.

Recorrido de redes: ¿Para que?

Recorrer la red que soporta la comunicación entre los procesos es importante para **Actualizar ó Recolectar** información mantenida localmente por cada uno de los procesos participantes en un algoritmo distribuido.

Aplicación: Difusión de información, asignación de identificación única a cada proceso, realizar un cálculo arborescente con una raíz determinada, etc.

Recorrido de redes: hipótesis

- El grafo **G** que modela la aplicación distribuida puede ser cualquiera, pero conexo y no dirigido
- Cada proceso posee una identificación única y es posible ordenarlos estrictamente de acuerdo a ella
- Ningún nodo conoce previamente la topología
- Cada nodo sólo conoce las líneas que lo conectan a sus vecinos, pero no necesariamente la identidad de sus vecinos
- Las líneas de comunicación son fiables.

Comunicación fiable

- No hay pérdida de mensajes
- No hay alteración de mensajes
- No hay duplicación de mensajes
- No hay alteración de secuencia
- Tiempo de recepción arbitrario pero finito.

Recorrido de redes: Enfoques

1. **Recorrido en profundidad**: Se considera un nodo privilegiado P_r y el objetivo es permitir a P_r realizar a través de mensajes, un recorrido en profundidad o secuencial del grafo asociado a la aplicación.
2. **Recorrido en paralelo**: Los nodos no tienen que ser visitados secuencialmente, ya que varios de ellos puedes ser visitados simultáneamente.

Recorrido en profundidad

Se basa en el marcaje de los nodos, siguiendo las reglas:

1. Cuando un nodo recibe un mensaje de *recorrido* por primera vez, se marca y re-transmite el mensaje a uno de sus vecinos
2. Cuando un nodo ya marcado recibe nuevamente un mensaje de recorrido, responde inmediatamente al emisor avisándole que él ya fué *visitado*
3. Cuando un nodo recibe la respuesta de un vecino, selecciona un nuevo vecino y le transmite el mensaje de *recorrido*
4. Si un nodo tiene la certeza de que todos sus vecinos están marcados, envía un mensaje al vecino que provocó su marca avisándole que ya fué *visitado*
5. Cuando el nodo iniciador $\mathbf{P_r}$ sepa que todos sus vecinos fueron *visitados*, determinará que la red fué recorrida.

Recorrido en profundidad: Algoritmo

Se transmiten dos tipos de mensajes de control:

- *Rec*: mensaje de recorrido
- *Vis*: mensaje de aviso de ya visitado

```
Var  marca :lógico inicializa falso;%indica si ya fue visitado o no
     Aux conjunto de Procesos inicializa Vecinos;
     padre: Procesos; %Proceso que originó la visita a Id

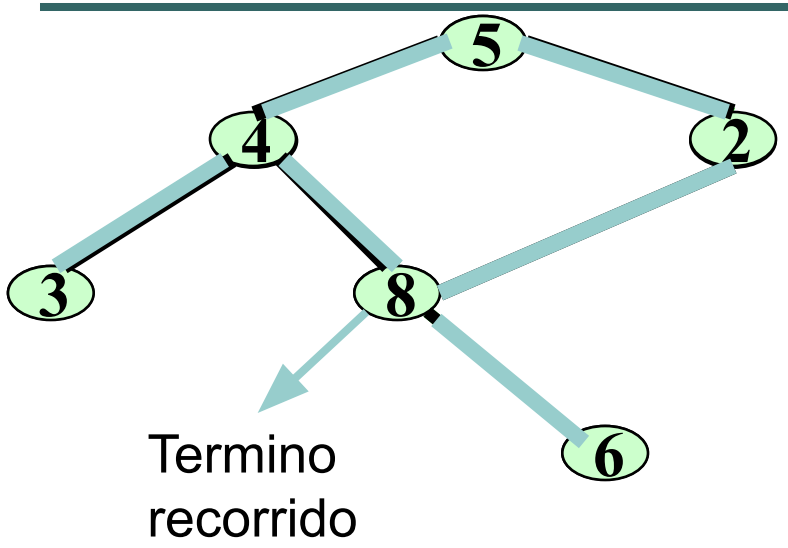
% Para iniciar un recorrido. Sólo para el proceso Pr.
Si Id = r entonces marca:= verdad;
           Enviar(Rec) a Un-Elemento{Aux};
fsi;

% proceso Un-Elemento(A: conjunto de procesos), selecciona y retorna %
un elemento del conjunto
```

Recorrido en profundidad: Algoritmo

```
Cuando llegue un mensaje  hacer
Recibir (tipo) de emisor;
Aux:= Aux - {emisor};
Selección
    tipo=Rec: Si marca entonces Enviar(Vis) a emisor;
          sino marca:=verdad; padre:=emisor;
            Si Vacio{Aux} entonces Enviar(Vis) a padre;
            sino    Enviar(Rec) a Un-Elemento{Aux};
            fsi;
          fsi;
    tipo=Vis: Si Vacio{Aux} entonces
                Si Id=r entonces % terminó recorrido
                sino Enviar(Vis) a padre;
                fsi;
          sino Enviar(Rec) a Un-Elemento{Aux};
          fsi;
fselección;
fin
```

Recorrido de redes: Ejemplo Pr = 8



Aux (3) = {4}, Padre(3) = 4
Aux(4) = {3,5,8} Padre(4) = 5
Aux(5) = {2,4} Padre(5) = 2
Aux(2) = {5,8} Padre(2) = 8
Aux(8) = {2,4,6}
Aux(6) = {8} Padre(6) = 8

Mensajes Rec Mensajes Vis

Recorrido en paralelo

La solución en este caso consiste en que cuando un nodo P_i recibe un mensaje de recorrido por primera vez, lo propaga a TODOS sus vecinos, excepto al emisor del mensaje, y no espera por la respuesta de ellos.

Esto garantiza que P_i no vuelva a recibir mensajes de recorrido.

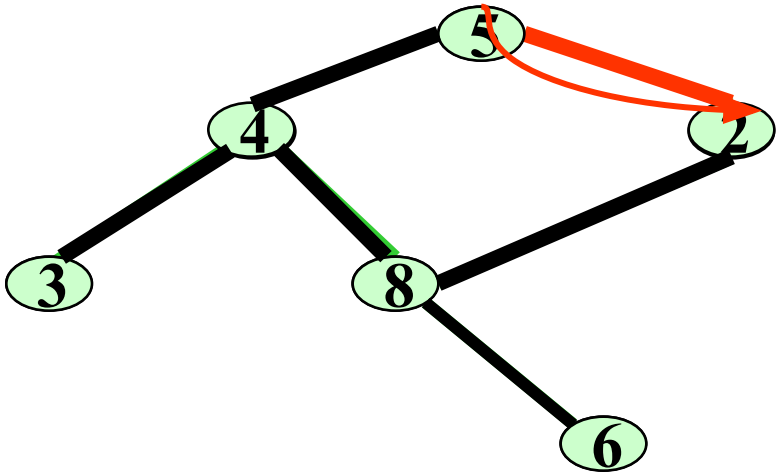
Recorrido en paralelo: Algoritmo

Se transmiten un sólo tipo de mensaje de control: *Rec*: mensaje de recorrido

```
Var marca :lógico inicializa falso; %indica si ya fue visitado o no
    padre : Procesos; %Proceso que originó la visita a Id

% Para iniciar un recorrido. Sólo para el proceso Pr.
Si Id = r entonces
        marca:= verdad;
        paratodo {j ∈ Vecinos} hacer
            Enviar(Rec) a  j;
        fparatodo;
Sino
    Cuando llegue un mensaje  hacer
    Recibir (tipo) de emisor;
    Si Not marca entonces marca:=verdad; padre:=emisor;
      paratodo {j∈Vecinos / j≠padre} hacer Enviar(Rec)a j;
fparatodo;
    fsi;
Fsi;  fin
```

Recorrido de redes: Ejemplo Pr = 8



Vecinos(3) = {4}
Vecinos(4) = {3,5,8}
Vecinos(5) = {2,4}
Vecinos(2) = {5,8}
Vecinos(8) = {2,4,6}
Vecinos(6) = {8}

Mensaje Rec

Recorrido en profundidad Vs. Recorrido en paralelo

1. En el recorrido en paralelo el nodo emisor no tiene conocimiento de la finalización del proceso
2. En el recorrido en profundidad el hecho de que el conjunto Aux sea vacío, le indica al emisor que todos sus vecinos fueron visitados y se puede dar por terminado el recorrido
3. Recorrido en paralelo permite difundir información, pero no recolectar.

